

Introduzione ad EcmaScript 2015

Con ecmaScript 2015 o es6 sono state introdotte nuove funzionalità in javascript :

- let-const(block scope)
- template literal
- arrow function
- destructuring array and objects
- spread e rest operator
- object assign
- array map-filter-find
- classes and inheritance

Lato server:

- async await
- promises
- fetch api

Let vs Var

La differenza più necessaria in fase di sviluppo è stata di definire a livello di scope una variabile con let o costante con const, ovvero la variabile o costante viene vista dal compilatore solo nel blocco laddove è dichiarata rispetto a var in cui non aveva visibilità block scope.

Template Literal

Il template literal permette di avere un codice più pulito quando si devono concatenare o utilizzare più stringhe dove prende il nome di string interpolation.

Esempio:

prima

adesso

```
Consol.log(nome+nazione)
```

```
console.log(`${nome} ${nazione}`)
```

Tutto all'interno del backtick(alt+96) mi permette di non dover concatenare o unire più dati.

Arrow function

Un arrow function è sempre una funzione come quelle viste nei moduli precedenti ma con meno codice da dover scrivere, quindi utilizzando una sintassi più rapida.

Esempio

function

```
Function myFunc(nazione){  
  return nazione ;  
  console.log(myFunc('italia'))  
}
```

arrow function

```
let myFunc =(nazione)=>  
  nazione;
```

```
let func = myFunc('italia')  
console.log(func)
```

Destructuring array and object

La destrutturazione si può applicare a strutture di dati oppure oggetti e consente di estrarre informazioni a dati singoli applicati ad una struttura.

Esempio

array

```
let nome,nazione;
```

```
[nome,nazione] = ['marco','italia'];  
console.log(nome)  
console.log(nazione)
```

```
let oggetto = {nome:'marco',nazione:'italia'};  
  
let {nome,nazione} = oggetto  
  
console.log(`${nome} ${nazione}`)
```

Spread operator

La Spread consente un iterabile come un'espressione di un array o una stringa da espandere in punti in cui sono previsti zero o più argomenti (per chiamate di funzione) o elementi (per letterali di array) o un'espressione di oggetto da espandere in posizioni dove zero o più sono previste coppie di valori (per valori letterali oggetto).

```
let arr = [1,2,3]
let arrs= [4,5,6]
let spread = [...arr...arrs]
console.log(spread)
```

Array map

Nel seguente esempio avremo un arrow function che dividerà per due ogni elemento dell'array caratterizzato dal metodo map

```
let arr =[10,20,30];
let map = arr.map(result=>result/2);
console.log(map)
```

Array filter

Nel seguente esempio avremo una function che restituirà tutti gli elementi dell'array maggiori di 6 utilizzando il metodo filter.

```
let arr = [6,8,11,15]
let filter = arr.filter(function(number) {
  return number > 7;
});
```

Array find

Nel seguente esempio avremo una function che restituirà il primo elemento dell'array maggiore di 215 utilizzando il metodo find.

```
function finder(element) {

  return element > 215;

}

let finds= [120, 500, 800, 130, 440].find(finder);
console.log(finds)
```